DEVORISE AI

Sales Pricing Matrix

Software Development Life Cycle (SDLC)

Document Version: 1.0

Date: March 9, 2026

Project Owner: Devorise AI

Prepared By: Engineering Team

Classification: Internal — Confidential

Table of Contents

1. **Stage 1** — Planning & Requirements
2. **Stage 2** — System Analysis
3. **Stage 3** — System Design
4. **Stage 4** — Implementation
5. **Stage 5** — Testing & Quality Assurance
6. **Stage 6** — Deployment & Maintenance

STAGE 1

Planning & Requirements

1.1 Project Overview

The **Sales Pricing Matrix** is an internal SaaS tool for Devorise AI's sales team. It enables sales architects to configure AI transformation proposals for clients by selecting industry-specific modules, infrastructure options, and add-ons — then generate a branded PDF proposal in one click.

1.2 Business Objectives

ID	OBJECTIVE
O1	Eliminate manual proposal creation (~2 hours per proposal → under 5 minutes)
O2	Standardize pricing across all sales architects
O3	Deliver professional, branded PDF proposals to clients
O4	Support multi-currency (JOD/USD) and tax calculations
O5	Expand into a full platform with user management and customization

1.3 Stakeholders

ROLE	RESPONSIBILITY
Product Owner	Defines features, priorities, and acceptance criteria
Sales Architects	Primary users — create and send proposals to clients
Engineering Team	Builds and maintains the platform
Clients	Receive generated PDF proposals

1.4 Functional Requirements

ID	REQUIREMENT	PRIORITY
FR-01	User enters client company name, contact person, and role	P0
FR-02	User selects an industry vertical (e.g., AI R&D, FinTech, Healthcare)	P0
FR-03	System displays industry-specific AI modules with setup and monthly pricing	P0
FR-04	User selects/deselects modules; sidebar updates totals in real-time	P0
FR-05	User toggles currency between JOD and USD	P0
FR-06	User toggles 16% tax on/off	P0
FR-07	User selects infrastructure type (Integration Only / System Creation)	P1
FR-08	User selects communication channels (WhatsApp, Website, Social Media, Internal)	P1
FR-09	User selects hosting provider (Devorise AI / Client AWS/GCP)	P1
FR-10	User enables Bulk Messaging add-on (+100 JOD/mo)	P1
FR-11	System calculates One-Time Setup, Monthly Recurring, and Yearly totals	P0
FR-12	System shows an Efficiency Impact meter based on selected modules	P1
FR-13	User clicks "Download Proposal PDF" → branded PDF downloads	P0
FR-14	PDF includes: Cover page, Solution, Roadmap, Pricing table, Terms, Agreement	P0
FR-15	**Landing Page**: Public-facing page with hero, features, pricing tiers	P1
FR-16	**Profile Management**: Login/register, user profiles, saved proposal history	P1
FR-17	**Customization**: Custom branding (logo, colors), PDF template selection, module builder	P2

1.5 Non-Functional Requirements

ID	REQUIREMENT
NFR-01	Page load time under 2 seconds
NFR-02	Works on Chrome, Firefox, Edge, Safari
NFR-03	Responsive design (desktop, tablet, mobile)
NFR-04	Dark theme with glassmorphism aesthetic matching Devorise brand
NFR-05	PDF generation entirely client-side (no server dependency for MVP)
NFR-06	Zero external API dependencies for core pricing functionality

1.6 Constraints

- MVP is a **static single-page app** (no backend for phase 1)
- PDF generation uses **jsPDF** (client-side, no server rendering)
- All pricing data is hardcoded in `pricing.js` for MVP
- Brand assets (logos) are embedded as Base64 in `brand-assets.js`

STAGE 2

System Analysis

2.1 Current System State

The tool operates as a **static single-page web application**:

```
User opens browser → Loads index.html → Selects options → JS calculates pricing → jsPDF generates PDF → Browser downloads
```

There is no backend, no database, and no authentication. All logic runs in the browser.

2.2 Data Model — Pricing Configuration

```
PRICING_CONFIG
├── exchangeRates: { JOD: 1, USD: 1.41 }
├── taxRate: 0.16
└── industries: {
      [INDUSTRY_KEY]: {
```

```
        id: string
        name: string
        theme: string (hex color)
        narrative: string
        roi: string
        modules: [
          {
            id: string
            name: string
            setup: number (JOD)
            monthly: number (JOD)
            description: string
            efficiency: number (0-100)
          }
        ]
      }
    }
```

2.3 Application State

```
state = {
    industry: Industry | null
    selectedModules: Module[]
    currency: 'JOD' | 'USD'
    isTaxEnabled: boolean
    clientName: string
    salespersonName: string
    clientNarrative: string
    totalSetup: number
    totalMonthly: number
    totalYearly: number
    impactScore: number (0-95)
}
```

2.4 Calculation Logic

```
totalSetup    = SUM(selectedModules.setup)
totalMonthly  = SUM(selectedModules.monthly)
totalYearly   = totalSetup + (totalMonthly × 12)
impactScore   = MIN(95, SUM(selectedModules.efficiency))

If currency == 'USD':  amount × 1.41
If tax enabled:        amount × 1.16
```

2.5 PDF Structure (6 Pages)

PAGE	CONTENT
1. Cover	Devorise logo, client name, "Agentic AI Integration Proposal", date, reference number
2. Proposed Solution	List of selected modules with descriptions
3. Implementation Roadmap	3 phases: Strategy & Integration, Agent Deployment, Monitoring
4. Investment & Pricing	Auto-generated pricing table (setup, monthly, yearly per module)
5. Terms & Conditions	Proposal validity, assumptions, responsibilities
6. Agreement & Signatures	Signature blocks for Devorise and client

STAGE 3

System Design

3.1 Architecture

Current (Phase 1 — MVP)

```
Static SPA → served via http-server → all client-side JavaScript
```

Target (Phase 3+ — Full Stack)

```
┌──────────────── Frontend (Vite SPA) ────────────────┐
│  Landing Page │ Dashboard │ Builder │ History │ Settings │
└──────────────────────────────────────────────────────┘
                          │
                          │ REST API
                          │
┌──────────────────────────────────────────────────────┐
│           Backend (Express + TypeScript)             │
│  Auth │ Users │ Proposals │ Clients Routes           │
└──────────────────────────────────────────────────────┘
                          │
                          │
                ┌─────────────────────┐
                │  SQLite / PostgreSQL │
                └─────────────────────┘
```

3.2 File Structure

```
Project 1/
        ├── index.html # Main pricing matrix page
        ├── style.css # All CSS (dark glassmorphism theme)
        ├── app.js # SalesOS class — UI logic & state management
        ├── pricing.js # PRICING_CONFIG — industries, modules, rates
        ├── pdf-generator.js # ProposalGenerator — jsPDF document builder
        ├── brand-assets.js # Base64-encoded logos and images for PDF
        ├── logo.png # Devorise AI logo for web header
        │
        ├── backend/ # (Future) Express API
        │ ├── package.json
        │ └── tsconfig.json
        │
        └── frontend/ # (Future) Vite SPA
        ├── package.json
        ├── vite.config.ts
        └── src/
```

3.3 UI Design Specifications

ELEMENT	SPECIFICATION
Background	`#0F1014` (dark navy)
Card Surfaces	`#1A1B21` with `rgba(255,255,255,0.08)` border
Accent Color	`#007C8A` (teal) — buttons, highlights, active states
Typography	Inter (Google Fonts), weights 300–800
Border Radius	Cards: 16px, Inputs: 12px, Pills: 20px
Layout	CSS Grid, 2-column (form left + sticky sidebar right)
Animations	fadeInUp on cards, cubic-bezier(0.16, 1, 0.3, 1) transitions

3.4 Component Map

```
index.html
        ├── header
        │ ├── brand (logo)
        │ ├── control-panel (currency toggle, tax toggle)
        │ ├── h1 "Sales Pricing Matrix"
```

```
          │ └── subtitle
          │
          ├── main.main-content
          │ ├── section.form-section
          │ │ ├── card: Client Information
          │ │ │ ├── Company Name input
          │ │ │ ├── Contact Person input
          │ │ │ ├── Title/Role input
          │ │ │ ├── Industry Vertical grid
          │ │ │ └── Brief Project Description textarea
          │ │ │
          │ │ ├── card: 01 Infrastructure
          │ │ │ ├── Platform Type radio cards
          │ │ │ └── Channel pills
          │ │ │
          │ │ ├── card: 02 AI Intelligence & Scope (hidden until industry selected)
          │ │ │ └── Module list (dynamically rendered from pricing.js)
          │ │ │
          │ │ └── card: 03 Hosting & Add-ons
          │ │ ├── Hosting Provider radio
          │ │ └── Bulk Messaging toggle
          │ │
          │ └── aside.receipt-section (sticky sidebar)
          │ └── receipt-card
          │ ├── "Final Quote Summary"
          │ ├── ONE-TIME SETUP (items + total)
          │ ├── MONTHLY RECURRING (total)
          │ ├── YEARLY COMMITMENT (total)
          │ ├── API Usage note
          │ ├── Efficiency Impact meter
          │ ├── View Company Profile button
          │ └── Download Proposal PDF button
          │
          └── scripts: brand-assets.js, app.js (ES module)
```

STAGE 4

Implementation

4.1 Tech Stack

LAYER	TECHNOLOGY	PURPOSE
Markup	HTML5	Structure and semantics
Styling	Vanilla CSS3	Dark theme, glassmorphism, responsive grid
Logic	Vanilla JavaScript (ES Modules)	State management, DOM manipulation, events
PDF	jsPDF + jspdf-autotable	Client-side PDF generation
Icons	Font Awesome 6	UI icons
Fonts	Google Fonts (Inter)	Typography
Dev Server	http-server (npx)	Local static file serving
Backend (future)	Express.js + TypeScript	REST API
Database (future)	SQLite → PostgreSQL	Data persistence

4.2 Core Files — Implementation Guide

`pricing.js` — Data Layer (Build First)

Contains all pricing configuration. Exports `PRICING_CONFIG` (industries, exchange rates, tax rate) and `GLOBAL_PHASES` (implementation roadmap phases). Each industry has modules with setup cost, monthly cost, description, and efficiency score.

`app.js` — Application Logic (Build Second)

Contains the `SalesOS` class with methods:

METHOD	PURPOSE
`constructor()`	Initializes state object, calls `init()`
`init()`	Calls `renderIndustries()` and `bindEvents()`
`renderIndustries()`	Injects industry cards into DOM from config
`bindEvents()`	Attaches all click/input/change event listeners
`selectIndustry(key, card)`	Sets industry, reveals modules, applies theme color
`renderModules()`	Injects module list for selected industry
`toggleModule(id)`	Adds/removes module from selection
`setCurrency(curr)`	Switches between JOD and USD
`formatVal(amount)`	Applies exchange rate + tax, formats as currency
`calculateTotals()`	Computes setup, monthly, yearly, impact score
`render()`	Updates all DOM elements with current state
`handleGenerate()`	Dynamically imports pdf-generator.js, triggers PDF

`pdf-generator.js` — PDF Generation (Build Third)

Contains the `ProposalGenerator` class. Receives app state, builds a 6-page PDF using jsPDF with branded header/footer, cover page, solution listing, roadmap, pricing table (autoTable plugin), terms, and signature blocks.

`brand-assets.js` — Brand Assets

Base64-encoded images (logo, banners) for the PDF. Avoids external image loading issues in jsPDF.

`style.css` — Styling

CSS custom properties define the entire design system. Key variables: `--devorise-navy: #0F1014`, `--devorise-card: #1A1B21`, `--devorise-teal: #007C8A`.

4.3 How to Run Locally

```
# 1. Clone the repository
git clone <repo-url>
cd "Project 1"

# 2. Start a local server (ES Modules require HTTP, not file://)
npx -y http-server . -p 5000 --cors -c-1
```

```
# 3. Open in browser
# Navigate to http://localhost:5000
```

4.4 Adding a New Industry

Edit `pricing.js` — add a new key to `PRICING_CONFIG.industries`. No other file changes needed; the UI renders dynamically from this config.

```
NEW_INDUSTRY: {
    id: "new_industry",
    name: "Industry Display Name",
    theme: "#HEX_COLOR",
    narrative: "What the AI solution does for this industry.",
    roi: "Expected XX% improvement in [metric].",
    modules: [
        {
            id: "module1",
            name: "Module Name",
            setup: 2000,        // JOD
            monthly: 500,       // JOD
            description: "What this module does.",
            efficiency: 40      // 0-100 impact score
        }
    ]
}
```

STAGE 5

Testing & Quality Assurance

5.1 Test Strategy

TEST TYPE	SCOPE	METHOD
Functional	All FR-01 through FR-14	Manual browser testing
UI / Visual	Design matches specifications	Screenshot comparison
Calculation	Pricing math accuracy	Manual verification
PDF	PDF content and formatting	Download and inspect
Cross-Browser	Chrome, Firefox, Edge, Safari	Manual testing
Responsive	Desktop, tablet, mobile	Browser DevTools

5.2 Test Cases

TC	DESCRIPTION	STEPS	EXPECTED RESULT
TC-01	Page loads	Open localhost:5000	No JS errors, all sections visible
TC-02	Client info	Type in all 3 fields	Values stored in state
TC-03	Industry selection	Click an industry card	Card highlights, modules appear, narrative fills
TC-04	Module selection	Click 2 modules	Both highlight, sidebar shows prices
TC-05	Pricing accuracy	Select modules (1600+1800)	Setup=3400, Monthly=1000, Yearly=15400
TC-06	Currency toggle	Click USD	All prices × 1.41
TC-07	Tax toggle	Enable 16% tax	All prices × 1.16
TC-08	USD + Tax	Enable both	Prices × 1.41 × 1.16
TC-09	Module deselect	Click selected module again	Deselects, prices decrease
TC-10	PDF generation	Fill name, select modules → Download	PDF downloads with 6 pages
TC-11	PDF content	Open downloaded PDF	Cover shows client name, pricing matches
TC-12	Empty state	No modules selected	Sidebar shows placeholder text
TC-13	Missing name	Download without company name	Alert: "Please specify the Client Name"

5.3 Known Limitations (MVP)

ISSUE	IMPACT	NOTES
System Creation (+5000 JOD) not added to total	Low	Radio selection not wired to pricing logic yet
Bulk Messaging add-on not added to total	Low	Toggle not wired to pricing logic yet
Channel pills are visual only	Low	No pricing impact
View Company Profile opens nothing	Low	Needs target URL or PDF

STAGE 6

Deployment & Maintenance

6.1 Deployment Options

Option A: Static Hosting (Current — MVP)

```
# Serve from any static host (Netlify, Vercel, GitHub Pages, S3)
npx -y http-server . -p 5000
```

Pros: Zero infrastructure, instant deploy, free hosting
Cons: No user accounts, no saved data

Option B: Full Stack (Future)

```
# Docker Compose
docker-compose up -d

# Services:
# - frontend: Vite build → Nginx
# - backend: Express API on port 3000
# - db: PostgreSQL on port 5432
```

6.2 Environment Configuration (Future)

```
# .env
PORT=3000
DATABASE_URL=postgresql://user:pass@localhost:5432/devorise
JWT_SECRET=<random-secret>
NODE_ENV=production
```

6.3 CI/CD Pipeline (Future)

```
Git Push → GitHub Actions → Lint & Type Check → Run Tests → Build Frontend → Build Docker Image → Deploy to Server
```

6.4 Maintenance Checklist

TASK	FREQUENCY
Update pricing data in `pricing.js`	As needed
Add new industry modules	Per client onboarding
Update brand assets (logo, colors)	Quarterly
Dependency security updates	Monthly
Cross-browser testing	Per release
Backup database (when implemented)	Daily

6.5 Version History

VERSION	DATE	CHANGES
1.0	March 2026	Initial MVP — pricing matrix + PDF generation
1.1	Planned	Landing page + interface overhaul
2.0	Planned	Profile management + customization engine

Quick Start for New Team Members

```
# 1. Clone and enter project
git clone <repo-url>
cd "Project 1"

# 2. Start local server
npx -y http-server . -p 5000 --cors -c-1

# 3. Open http://localhost:5000

# Key files to understand (in order):
#   pricing.js        → Data: industries, modules, pricing
#   index.html        → UI: layout, forms, sidebar
#   style.css         → Design: dark theme, glassmorphism
#   app.js            → Logic: state management, events, rendering
#   pdf-generator.js  → PDF: jsPDF document builder
#   brand-assets.js   → Assets: Base64 logos for PDF
```

> **Rule**: All pricing changes go in `pricing.js`. All UI changes go in `index.html` + `style.css`. All logic changes go in `app.js`. Keep them separate.